LambdaScript : A Functional Programming Language

Alex Kozik
Cornell University
Ithaca, NY
ajk333@cornell.edu

***Abstract*—We present LambdaScript, a statically-typed functional programming language inspired by Haskell and OCaml. LambdaScript features Hindley-Milner type inference, polymorphic algebraic data types, comprehensive pattern matching, and first-class functions with closures. The language supports advanced features including mutually recursive type definitions, list comprehensions, and operators as first-class values. This paper describes both the formal semantics and the complete implementation of LambdaScript, including the design of the type system, operational semantics, and the architecture of a full interpreter pipeline consisting of lexical analysis, parser combinators, constraint-based type inference, and environment-based evaluation. The implementation is validated through 1200 unit tests covering complex real-world algorithms including red-black trees, interpreters, and graph traversal.**

***Index Terms*—functional programming, type inference, algebraic data types, pattern matching, programming languages, formal semantics**

I. Introduction

Statically-typed functional programming languages such as Haskell, OCaml, and Standard ML provide powerful abstractions for writing correct programs through algebraic data types, pattern matching, and polymorphic type inference. The Hindley-Milner type system enables programmers to write code without explicit type annotations while maintaining complete type safety through automatic type inference. However, understanding how these features interact—both formally through typing rules and operationally through implementation—requires navigating complex, mature language ecosystems with decades of accumulated features and optimizations.

This paper presents **LambdaScript**, a functional programming language designed to demonstrate that rigorous formal semantics and practical implementation can coexist harmoniously. LambdaScript features a complete Hindley-Milner type inference system, polymorphic algebraic data types, comprehensive pattern matching, and advanced features including mutually recursive type definitions. Rather than merely describing theoretical properties, we provide both a complete formal semantics and a fully implemented interpreter validated through extensive testing.

A key contribution of LambdaScript is its treatment of **mutually recursive algebraic data types**, which enable natural representation of complex data structures such as trees with forests, abstract syntax trees with multiple node types, and graph-based structures. While languages like OCaml and Haskell support such types, their formal semantics and implementation present subtle challenges in constraint generation, type environment management, and ensuring that recursive references are properly resolved during type checking. LambdaScript addresses these challenges with a clean syntax using the `and` keyword and a carefully designed type checking algorithm that extends the standard Hindley-Milner approach.

The implementation consists of a complete interpreter pipeline: a lexer that tokenizes source code, a parser built using parser combinators for compositional grammar definitions, a constraint-based type checker implementing Hindley-Milner inference with extensions for recursive types, and an environment-based evaluator that handles closures, recursion, and pattern matching. The entire system is written in OCaml and validated through 1200 unit tests covering type inference edge cases, evaluation correctness, and real-world algorithms including red-black tree operations.

The main contributions of this paper are:

- A formal semantics for a functional language with mutually recursive algebraic data types, including complete typing rules and operational semantics
- A complete implementation architecture demonstrating how theory translates to practice, including lexer, parser combinators, constraint-based type inference, and environment-based evaluation
- Extensions to the standard Hindley-Milner algorithm to support mutually recursive types and proper handling of fixed-point types
- Validation through 1200 unit tests demonstrating correctness on complex algorithms including red-black trees, showcasing the language's expressiveness

The remainder of this paper is organized as follows. Section II presents examples demonstrating LambdaScript's features, building intuition before formal treatment. Section III describes the type system design, including polymorphism, algebraic data types, and type inference. Section IV provides detailed formal semantics with typing rules and operational semantics. Section V details the implementation architecture of each pipeline component. Section VI presents testing methodology and evaluation results. Section VII concludes with reflections and future directions.

II. Examples

This section presents examples that demonstrate LambdaScript's core features, building intuition for the formal treatment in subsequent sections. We showcase algebraic data types, pattern matching, polymorphism, and higher-order functions through practical examples.

A. Polymorphic List Data Type

We begin by defining a polymorphic list type using recursive algebraic data types. LambdaScript uses the `type rec` keyword for recursive type definitions and the `'a` syntax for type variables, following OCaml conventions.

```
type rec List<'a> =
  | Nil
  | Cons of ('a, List<'a>)
```

This definition introduces two constructors: `Nil` represents the empty list, and `Cons` takes a tuple containing an element of type `'a` and the rest of the list. The type parameter `'a` makes this list polymorphic, allowing lists of any type.

With the list type defined, we can implement fundamental higher-order functions that operate over lists.

1) Map Function: The `map` function applies a function to each element of a list:

```
let rec map f lst =
  case lst do
  | Nil -> Nil
  | Cons (h, t) -> Cons (f h, map f t)
```

The type system infers the polymorphic type:

```
map : ('a -> 'b) -> List<'a> -> List<'b>
```

This demonstrates parametric polymorphism: `map` works for any input type `'a` and output type `'b`, as long as the function `f` has type `'a -> 'b`.

Example usage:

```
let double = fn x -> x * 2 in
let numbers = Cons (1, Cons (2, Nil)) in
map double numbers
(* Result: Cons (2, Cons (4, Nil)) *)
```

2) Reduce Function: The `reduce` function (also known as fold) combines list elements using a binary operator:

```
let rec reduce op acc lst =
  case lst do
  | Nil -> acc
  | Cons (h, t) -> reduce op (op acc h) t
```

Inferred type: `reduce : ('b -> 'a -> 'b) -> 'b -> List<'a> -> 'b`

The type signature shows that `reduce` can produce a result of a different type (`'b`) than the list elements (`'a`), demonstrating the power of polymorphic type inference.

Example summing a list:

```
let add = fn x -> fn y -> x + y in
let numbers = Cons (1, Cons (2, Nil)) in
reduce add 0 numbers
(* Result: 3 *)
```

B. Option Monad

The Option type represents computations that may fail, providing a type-safe alternative to null values. We define it as a sum type with two constructors:

```
type Option<'a> =
  | None
  | Some of 'a
```

`None` represents absence of a value, while `Some` wraps a present value of type `'a`.

1) Monadic Bind Operation: The bind operation chains optional computations. In LambdaScript, we can define it as an operator:

```
let (>>=) opt f =
  case opt do
  | None -> None
  | Some x -> f x
```

Inferred type: `(>>=) : Option<'a> -> ('a -> Option<'b>) -> Option<'b>`

This type signature is the hallmark of monadic bind: it takes an `Option<'a>`, a function from `'a` to `Option<'b>`, and produces `Option<'b>`. If the input is `None`, the computation short-circuits; otherwise, the function is applied to the unwrapped value.

2) Return Operation: The `return` operation (called `pure` in some languages) lifts a value into the Option context:

```
let return x = Some x
```

Inferred type: `return : 'a -> Option<'a>`

3) Map for Option: Mapping a function over an optional value:

```
let map_opt f opt =
  case opt do
  | None -> None
  | Some x -> Some (f x)
```

Inferred type: `map_opt : ('a -> 'b) -> Option<'a> -> Option<'b>`

4) Example: Safe Division: These operations enable safe computation chains. Consider safe division:

```
let safe_div x y =
  if y == 0 then None
  else Some (x / y)
in

let compute x y z =
  (safe_div x y) >>= (fn result1 ->
    (safe_div result1 z) >>= (fn result2 ->
      return result2))
in
```

```
compute 100 10 2
(* Result: Some 5 *)

compute 100 0 2
(* Result: None *)
```

The first computation succeeds: $100/10/2 = 5$. The second short-circuits at division by zero, returning `None` without attempting the second division. This demonstrates how the Option monad handles errors compositionally through pattern matching and type safety, without exceptions or null pointer errors.

These examples illustrate several key features of LambdaScript: algebraic data types enable clean data modeling, pattern matching provides exhaustive case analysis, parametric polymorphism allows generic code, and the type system infers complex types including higher-rank polymorphism in monadic operations.

III. Type System

LambdaScript features a Hindley-Milner type system with support for parametric polymorphism, algebraic data types, and mutually recursive type definitions. The type system is both sound and complete: every well-typed program has a principal type, and type inference always succeeds when a valid typing exists.

A. Basic Types and Type Inference

LambdaScript includes primitive types (`int`, `float`, `bool`, `string`, `char`, `unit`), composite types (functions, lists, tuples), and user-defined algebraic data types. The type checker automatically infers types without requiring explicit annotations:

```
let x = 42
(* Inferred: x : int *)

let add x y = x + y
(* Inferred: add : int -> int -> int *)

let id x = x
(* Inferred: id : 'a -> 'a *)
```

The identity function `id` receives the polymorphic type `'a -> 'a`, where `'a` is a type variable that can be instantiated to any type. This demonstrates **let-polymorphism**: functions bound with `let` can be used at multiple types in different contexts.

B. Type Variables and Generalization

Type variables represent unknown types during inference. When a function is defined with `let`, the type checker **generalizes** over all unconstrained type variables, introducing universal quantifiers. Consider:

```
let compose f g x = f (g x)
(* Type: ('b -> 'c) -> ('a -> 'b) -> 'a -> 'c *)
```

The type checker generates fresh type variables for `f`, `g`, and `x`, then analyzes the function application `f (g x)`:

- `g x` requires `g : 'a -> 'b` for some `'b`
- `f (g x)` requires `f : 'b -> 'c` for some `'c`
- The result has type `'c`

After constraint solving, the system generalizes to produce the most general type with three independent type variables.

C. Algebraic Data Types

Algebraic data types define new types using constructors. Each constructor may carry a payload:

```
type Result<'a, 'b> =
  | Ok of 'a
  | Error of 'b
```

When this definition is processed, the type checker:

1) Introduces `Result` as a new type constructor taking two type parameters
2) Creates constructor types: `Ok : 'a -> Result<'a, 'b>` and `Error : 'b -> Result<'a, 'b>`
3) Adds these constructors to the typing environment

Users can then construct and match on values:

```
let safe_sqrt x =
  if x < 0 then Error "negative"
  else Ok (sqrt x)
(* Type: int -> Result<float, string> *)

let handle result =
  case result do
  | Ok v -> v
  | Error msg -> 0.0
(* Type: Result<float, 'a> -> float *)
```

D. Recursive Types

Recursive types reference themselves in their definition. LambdaScript uses **isorecursive** semantics with explicit `type rec` declarations:

```
type rec Tree<'a> =
  | Leaf of 'a
  | Node of ('a, Tree<'a>, Tree<'a>)
```

Internally, this is represented as a fixed-point type μ`Tree`. The type checker ensures that:

- Each recursive occurrence is properly parameterized
- Constructor payloads are type-checked in an environment containing the recursive type
- Pattern matching correctly unfolds the recursive type

Example usage demonstrates automatic type inference through recursive structures:

```
let rec height tree =
  case tree do
  | Leaf _ -> 1
  | Node (_, l, r) ->
```

```
      1 + (if height l > height r
           then height l else height r)
(* Type: Tree<'a> -> int *)
```

E. Mutually Recursive Types

A key contribution of LambdaScript is clean support for mutually recursive type definitions using the `and` keyword. Multiple types can reference each other:

```
type rec Tree<'a> =
  | Leaf of 'a
  | Node of ('a, Forest<'a>)
and Forest<'a> =
  | Empty
  | Trees of (Tree<'a>, Forest<'a>)
```

The type checker processes mutually recursive types by:

1) Adding all type names to the environment simultaneously before processing any constructors
2) Type-checking each constructor's payload in the extended environment
3) Ensuring all type parameters are consistently applied across mutual references
4) Verifying that cycles are properly guarded by constructors (no infinite types)

This enables natural expression of complex data structures:

```
let rec tree_size t =
  case t do
  | Leaf _ -> 1
  | Node (_, forest) -> 1 + forest_size forest
and forest_size f =
  case f do
  | Empty -> 0
  | Trees (t, rest) ->
      tree_size t + forest_size rest
(* tree_size : Tree<'a> -> int
   forest_size : Forest<'a> -> int *)
```

The implementation extends the standard Hindley-Milner unification algorithm to handle fixed-point types and ensures that type variable substitutions propagate correctly across the mutual recursion group.

F. Type Annotations

While type inference eliminates most annotations, LambdaScript allows optional type annotations for documentation and disambiguation:

```
let f (x: int) : int = x + 1

let g (x: int) : int = x * 2
```

Type annotations are checked for consistency with inferred types. If an annotation conflicts with the inferred type, the type checker reports an error.

IV. Formal Semantics

A. Overview

We now present the formal semantics of LambdaScript, providing a rigorous mathematical foundation for both the type system and runtime behavior. Formal semantics serve multiple purposes: they enable precise reasoning about program behavior, provide unambiguous specifications for implementers, and allow formal verification of type soundness and other properties.

Our semantic framework consists of two complementary components:

Static Semantics defines the type system through typing judgments of the form $\Gamma \vdash e : \tau$, read as "under type environment Γ, expression e has type τ." The static semantics specify when programs are well-typed and determine the principal types of expressions through constraint-based type inference. We formalize the Hindley-Milner algorithm, including constraint generation, unification, and generalization. Special attention is given to the treatment of algebraic data types and mutually recursive type definitions, which require careful handling of type environments and fixed-point types.

Dynamic Semantics defines program evaluation through operational semantics. We use large-step (natural) operational semantics with evaluation judgments of the form $\langle e, \rho \rangle \Downarrow v$, indicating that expression e in environment ρ evaluates to value v. Large-step semantics directly relate expressions to their final values, making the evaluation rules more intuitive and closely aligned with the implementation. The dynamic semantics specify how values are computed, how functions are applied, how pattern matching proceeds, and how recursion is handled through environment-based closure semantics.

The static and dynamic semantics are designed to satisfy **type soundness**: well-typed programs do not get stuck during evaluation. This is formalized through the type preservation theorem: if $\Gamma \vdash e : \tau$ and $\langle e, \rho \rangle \Downarrow v$, then v has type τ. While we do not provide complete proofs here, the semantics are structured to facilitate such proofs.

We adopt standard mathematical notation: Γ denotes type environments mapping variables to types, ρ denotes value environments mapping variables to values, and τ ranges over types. Substitutions are written $[\tau'/\alpha]\tau$ for replacing type variable α with type τ' in type τ. Function types are written $\tau_1 \to \tau_2$, list types as $[\tau]$, and tuple types as $(\tau_1, \tau_2, \ldots, \tau_n)$.

B. Static Semantics

C. Dynamic Semantics